KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

Bachelor in Technology

in

Computer Science and Engineering

A Project Report On

Project CORTEX - Centralized Operational Responsive Technical EXecution System.An AI-powered desktop assistant designed to perform voice-controlled tasks, automate system operations, and enhance user productivity.

Submitted By

Srinath Neogi 22052940

Sayandeep Dey 22052753

Soumayan Ghosh 22052770

Soham Banerjee 22052335

Sarbeswar Nandi 22053190

Sangram Mandal 22053186

Under the Guidance of

Dr. Abhaya Kumar Sahoo

Associate Professor, CSE, KIIT-DU

2025 - 2026

Department of Computer Science and Engineering

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY (KIIT)

Deemed to be University U/S 3 of UGC Act, 1956

Certificate

This is to certify that the project work titled **"Project CORTEX – A desktop-based intelligent assistant powered by Artificial Intelligence and Machine Learning, designed to perform voice-controlled tasks, automate system operations, and enhance user productivity. "** submitted by:

Srinath Neogi 22052940

Sayandeep Dey 22052753

Soumayan Ghosh 22052770

Soham Banerjee 22052335

Sarbeswar Nandi 22053190

Sangram Mandal 22053186

has been carried out under my guidance and supervision in partial fulfillment of the requirements for the award of the **Bachelor of Technology** degree in **Computer Science and Engineering** at **KIIT University**, Bhubaneswar.

Project Guide: Dr. Abhaya Kumar Sahoo

Designation: Associate Professor, Dept. of CSE, KIIT-DU

Date: ___/ ___/ 2026

Acknowledgement

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have been responsible for the successful completion of this project work.

We express our sincere gratitude to **School of Computer Engineering, KIIT University** for providing us with a great opportunity to pursue our Bachelor's degree in this institution.

Special thanks to our guide **Dr. Abhaya Kumar Sahoo**, Associate Professor, for his guidance, which paved the way for the smooth progress of our project.

We also thank our family and friends for their constant support.

Srinath Neogi

Sayandeep Dey

Soumayan Ghosh

Soham Banerjee

Sarbeswar Nandi

Sangram Mandal

CONTENTS

1. **Introduction** 4-6

2. **Related Works** 7-11

3. **Methodology** 12-17

   3.1 Framework: System Architecture and Workflow Diagram

   3.2 Voice Processing: Speech Recognition and Text Conversion

   3.3 Intent Prediction: Machine Learning Model Training

   3.4 Action Module: Command Execution and Task Automation

   3.5 User Interface: Desktop UI Design and Interaction

4. **Results and Analysis** 18-20

5. **Future Work and Conclusion** 21

6. **References** 22

1. INTRODUCTION :

Background & Motivation

With the rapid advancement of **Artificial Intelligence (AI)** and **Machine Learning (ML)**, the way users interact with computer systems has significantly evolved. Traditional methods of interaction, such as manual input through keyboards and mice, can often be *time-consuming and inefficient*, especially when performing repetitive or system-level tasks.

In recent years, **voice-based assistants** have emerged as a powerful solution to enhance user experience by enabling **natural and hands-free interaction**. However, many existing systems are either limited in functionality, dependent on internet connectivity, or lack the ability to perform **custom system-level operations** efficiently and securely.

Users often face challenges in executing multiple tasks such as opening applications, retrieving system information, managing files, or controlling system settings. These tasks require manual navigation and technical knowledge, which can reduce productivity and create inefficiencies. Additionally, the absence of proper **security mechanisms** in many assistant systems raises concerns regarding **unauthorized access and misuse**.

To address these limitations, this project introduces a **desktop-based intelligent voice assistant**. The system integrates **speech recognition, machine learning-based intent prediction, and automated action execution** to provide a seamless and efficient user experience. Furthermore, it considers **basic security measures** to ensure that sensitive operations are protected from unauthorized access.

Problem Statement

Despite the availability of voice assistants, there are several challenges that limit their effectiveness in desktop environments:

- **Limited System Control:** - Most existing assistants lack the ability to perform **advanced system-level operations** such as file handling, application control, and system monitoring.
- **Inefficient Interaction Methods:** - Traditional interaction methods require **manual input and navigation**, which can be slow and inconvenient.
- **Lack of Customization:** - Many systems are not easily customizable to support **user-specific commands and workflows**.

- **Absence of Intelligent Command Understanding:** - Basic systems rely on predefined commands and fail to interpret **natural language variations** effectively.
- **Security Concerns:** - Voice assistants may execute sensitive commands without proper **authentication or authorization**. - This can lead to **unauthorized access, data exposure, or unintended system operations**.

This project aims to overcome these challenges by developing an intelligent system that can understand user commands, predict intent using **machine learning models**, execute tasks efficiently, and ensure **secure interaction**.

Project Objectives

The primary objectives of this project are:

1. **Develop a desktop-based intelligent voice assistant** capable of performing various system-level tasks using **voice commands**.
2. **Implement speech recognition techniques** to convert user voice input into text for further processing.
3. **Design and train a machine learning model** for accurate **intent prediction** based on user commands.
4. **Build an action execution module** that maps predicted intents to corresponding system operations such as opening applications, retrieving information, and controlling system settings.
5. **Incorporate basic security mechanisms** such as authentication or verification to prevent unauthorized access and ensure safe execution of commands.
6. **Create an interactive graphical user interface (GUI)** to provide real-time feedback and improve user experience.

Scope of the Project

This project encompasses the following components:

- **Voice Input Processing:** - Capturing user voice input through a microphone. - Converting speech into text using speech recognition techniques.

- **Intent Prediction using Machine Learning:** - Preprocessing textual commands. - Training and evaluating models to classify user intent accurately.
- **Task Automation and Execution:** - Implementing modules for executing system commands such as opening applications, fetching system information, and performing file operations.
- **Security and Access Control:** - Implementing basic authentication mechanisms (e.g., voice verification or password). - Ensuring that sensitive commands are executed only after proper validation.
- **User Interface Development:** - Designing a desktop-based graphical interface for interaction and visualization of assistant states (listening, processing, responding).
- **System Integration:** - Integrating all modules to create a unified and efficient assistant capable of real-time and secure interaction.

Naming Rationale: Project CORTEX

The name **CORTEX** is inspired by the ***cerebral cortex***, the part of the human brain responsible for intelligence, decision-making, and reasoning. The system reflects similar capabilities by analyzing user data and generating intelligent career recommendations.

The acronym **CORTEX** represents the core functionalities of the system:

- **C – Cognitive Intelligence:** The system understands user abilities, interests, and academic performance to build a meaningful profile.
- **O – Operational:** It ensures smooth and continuous functioning of all system processes, from data collection to recommendation delivery.
- **R – Responsive:** The system dynamically adapts to individual user inputs, preferences, and needs, providing personalized and timely career guidance.
- **T – Technical:** It leverages advanced technical methodologies, including machine learning models and data-driven analysis, to generate accurate career predictions.
- **EX – EXecution:** The system not only provides recommendations but also delivers them effectively, guiding users toward informed decision-making and real-world implementation.

2. RELATED WORKS :

1.A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition

The paper by Lawrence R. Rabiner explains **Hidden Markov Models (HMMs)** and their use in modeling real-world signals, especially in speech recognition. It describes how signals can be represented using statistical models, where HMMs are effective because they capture time-varying patterns through hidden states and observable outputs.

An HMM consists of states, observation symbols, transition probabilities, and initial probabilities. Unlike simple Markov models, the states are not directly observable but are inferred from output sequences.

The paper focuses on three key problems: computing the probability of an observation sequence, finding the most likely sequence of hidden states, and optimizing model parameters. Algorithms like the forward-backward method and Viterbi algorithm efficiently solve these problems.

It also discusses different HMM structures and extensions, including continuous models, making them suitable for complex applications. Overall, HMMs are shown to be powerful tools for analyzing sequential data and building practical systems like speech recognizers.

2. Enhanced AI Voice Assistance using Machine Learning and NLP

This paper presents an enhanced AI voice assistant that uses **machine learning (ML)** and **natural language processing (NLP)** to perform tasks through voice commands. The system aims to provide a user-friendly and efficient way to interact with computers, improving accessibility and productivity.

It addresses limitations of traditional voice assistants, such as poor contextual understanding and inaccurate responses, by improving intent recognition and maintaining conversation context. The assistant can better interpret user queries and deliver more relevant responses.

The system integrates APIs like OpenAI, Weather, and News to provide real-time information and services. Its workflow includes speech-to-text conversion, intent analysis, data retrieval, and voice response generation.

Overall, the approach demonstrates that combining ML, NLP, and API integration results in a more intelligent and effective voice assistant for modern applications.

3. Jarvis-Desktop Based Virtual Assistant

This paper presents **JARVIS**, a desktop-based virtual assistant designed to improve productivity through voice and text interaction. Built using Python, it integrates speech recognition, natural language processing (NLP), machine learning, and generative AI to automate routine tasks and enhance human-computer interaction.

The system addresses limitations of traditional desktop usage and existing assistants by providing a hybrid interface with both voice and chatbot modes. It supports features such as launching applications, managing schedules, controlling media, and retrieving real-time information. A Naive Bayes classifier is used for intent recognition, while the Gemini API acts as a fallback for handling complex queries.

The architecture includes multiple layers such as user interface, interaction, NLP processing, and data storage, ensuring modularity and scalability. The system also emphasizes privacy by running locally and storing user data securely.

Overall, the project demonstrates that combining AI, NLP, and automation techniques can create an efficient and intelligent desktop assistant, improving accessibility, user experience, and task management in modern computing environments.

4. Desktop Voice Assistant

This paper presents a **desktop voice assistant** that uses artificial intelligence (AI) and natural language processing (NLP) to enable users to interact with their computers through voice commands.

The assistant can perform various tasks such as opening applications, searching the internet, playing music, and executing basic commands. It works by capturing voice input, identifying keywords, and mapping them to predefined actions.

The development process includes identifying user needs, researching existing solutions, and refining the system based on feedback. Improvements such as better accuracy, faster performance, and enhanced user experience were incorporated.

Overall, the project demonstrates that integrating AI and NLP into desktop environments can create a simple, efficient, and user-friendly tool, enhancing productivity and making computer interactions more intuitive.

3. METHODOLOGY

3.1 Framework: System Architecture and Workflow

The methodology followed in our project, **CORTEX – Desktop Assistant**, is structured into several stages, starting from voice input to executing user commands. The system architecture is presented in Figure 1.

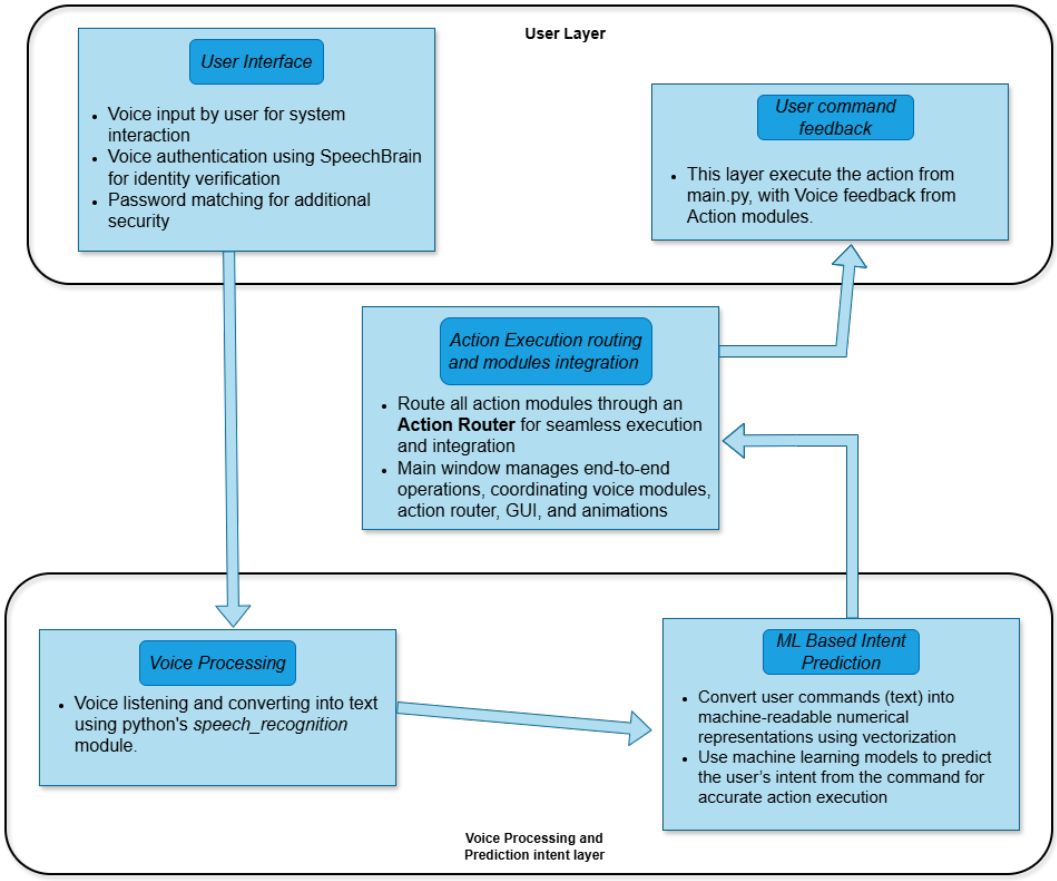

Figure 1: CORTEX System Architecture: From voice input to action execution.

The workflow of the system is as follows:

1. **Voice Input:** The user interacts with CORTEX through a microphone, giving voice commands.

2. **Voice Processing:** The audio signal is processed and converted to text using a Speech Recognition engine. Background noise is filtered, and optional voice authentication ensures only authorized users can interact.

3. **Intent Prediction:** The converted text is analyzed by a trained machine learning model to predict the intent of the user. The intent corresponds to a specific module or action.

4. **Action Execution:** The predicted intent is routed to the appropriate module, such as opening applications, fetching system information, performing internet searches, or executing scripts.

5. **User Feedback:** CORTEX provides visual and/or audio feedback to indicate the action performed, using a PyQt5-based desktop interface with animated visuals for listening, processing, and speaking states.

—

3.2 Voice Processing: Speech Recognition and Text Conversion

Voice processing in the CORTEX system is responsible for capturing user speech, converting it into text, and ensuring secure access through voice authentication. This module is implemented using Python-based libraries and custom classes.

1. Speech Recognition for Command Input

For general command processing, the system uses the `speech_recognition` library. A custom class named `Google_Listener` is implemented to handle real-time voice input.

The working of this module is as follows:

- The microphone captures the user's speech using `sr.Microphone()`.
- The recognizer listens for input with a timeout of 3 seconds and a maximum phrase duration defined by `listen_duration`.
- The captured audio is sent to the **Google Speech Recognition API** for conversion into text.
- The recognized text is returned for further processing.

To improve usability, the following parameters are configured:

- **Energy Threshold (300):** Helps detect speech over background noise.
- **Dynamic Energy Threshold:** Automatically adjusts sensitivity based on environment.
- **Pause Threshold (0.8):** Allows short pauses within speech without stopping recognition.

Error handling is also implemented to manage cases such as no speech detected, unclear audio, or API request failures.

2. Voice Authentication System

To ensure secure access, a dedicated voice authentication module is implemented using speaker recognition techniques and voice biometrics. This module operates independently from the general command listener and integrates multiple models for robust verification.

Enrollment Phase:

- The user is required to speak a predefined passphrase: *"its me srinath"*.
- A total of **five voice samples** are recorded using the `sounddevice` library.
- Each audio sample is stored as a `.wav` file with a sampling rate of 16 kHz.
- The `resemblyzer` library is used to preprocess audio and generate **voice embeddings**.
- Each embedding is normalized and stored in a NumPy array.
- All embeddings are saved in a file named `user_voice.npy` for future comparison.

Additionally, similarity between enrolled samples is computed using dot product to ensure consistency in voice recordings.

Authentication Phase:

- The user again speaks the same passphrase.
- A new voice sample is recorded using the `sounddevice` module.
- The recorded audio is processed using the **Whisper model** for accurate speech handling.
- The system generates a voice embedding for the new sample using the same embedding pipeline.
- The **SpeechBrain framework** is used for speaker verification by comparing the new embedding with stored embeddings.
- The system determines whether the speaker matches the enrolled user based on similarity scores.
- The access will be granted only if both Passphrase is matched and Voice Biometrics is passed.

Security and Failure Handling:

- The user is allowed a maximum of **three authentication attempts**.
- If all attempts fail, voice authentication is disabled temporarily.
- A **master password** option is provided as a fallback mechanism.
- A pop-up window prompts the user to enter the master password manually.
- If the master password is incorrect, the system determines unauthorized access and **locks the core system immediately**.

The figure shows the Architecture of Voice Authentication system in Project CORTEX. 2.

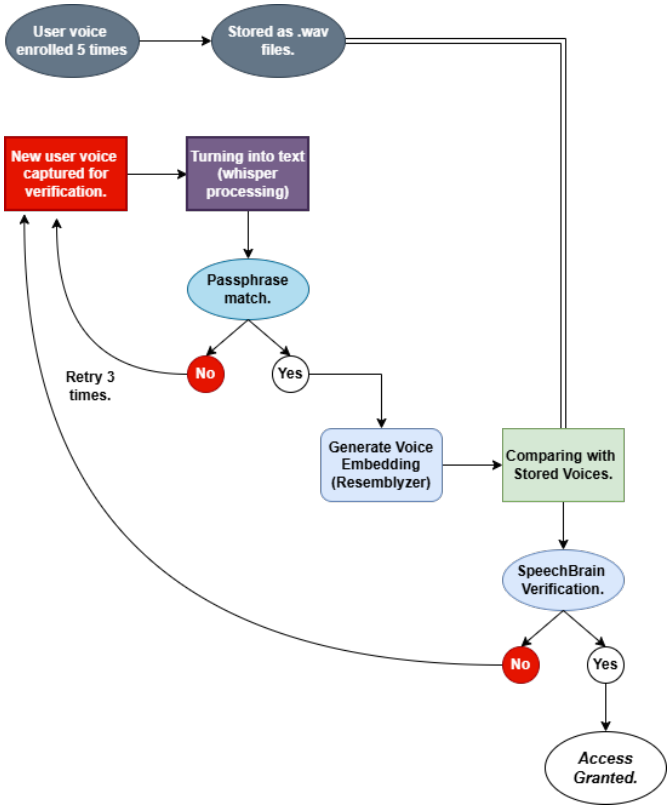

Figure 2: Voice Authentication Workflow in Project CORTEX.

3. Output to Next Module

After successful speech recognition and authentication, the processed text command is forwarded to the **Intent Prediction Module**, where the system determines the appropriate action to perform.

This approach ensures that CORTEX provides reliable voice interaction while maintaining system security through speaker verification. —

3.3 Intent Prediction: Machine Learning Model Training

Intent prediction is a core component of the CORTEX system, responsible for understanding user commands and mapping them to appropriate system actions. This is achieved using supervised machine learning models trained on a custom dataset of commands and corresponding intents.

1. Dataset Preparation

The dataset used for intent prediction consists of two primary columns:

- **Command:** Text-based user inputs such as *"hello"*, *"open chrome"*, *"battery status"*.
- **Intent:** Corresponding labels representing the action to be performed.

The dataset includes multiple intent categories such as:

- ***Intent Categories:***
  - **General:** greeting, quit
  - **Application & Web:** open_app, open_website
  - **System Controls:** brightness_control, volume_control, lock_screen, restart_pc, shutdown_pc
  - **System Monitoring:** battery_status, system_status, system_uptime, disk_status, wifi_status, internet_speed, ip_address
  - **File & Search:** doc_search, pdf_search, image_search, folder_search, ppt_search
  - **Utilities:** clipboard_read, date_time, screenshot
  - **Media Control:** media_control

This structured dataset enables the model to learn mappings between natural language commands and system-level actions.

2. Text Preprocessing and Feature Extraction

Since machine learning models cannot directly process raw text, the input commands are converted into numerical feature vectors using the **TF-IDF (Term Frequency–Inverse Document Frequency)** technique.

The implementation uses the `TfidfVectorizer` from `sklearn.feature_extraction.text` with the following configuration:

- **n-gram range (1,2):** Captures both single words and word pairs for better context understanding.
- **sublinear_tf=True:** Applies logarithmic scaling to term frequencies.
- **min_df=1:** Ensures all relevant words are included (important for small datasets).
- **max_df=0.95:** Ignores extremely common words that may not add value.

The output is a sparse matrix representing each command as a numerical vector, which is then used for model training.

3. Model Training and Evaluation

Two machine learning models were trained and evaluated for intent classification:

- **Logistic Regression** – Accuracy: **0.71**
- **Linear Support Vector Classifier (Linear SVC)** – Accuracy: **0.84**

The dataset was split into training and testing sets, and both models were evaluated based on their classification accuracy.

Among the two, **Linear SVC** demonstrated superior performance and better generalization, making it the preferred model for intent prediction in the final system.

4. Machine Learning Models Overview

Logistic Regression: Logistic Regression is a linear classification algorithm that predicts the probability of a given input belonging to a particular class. It is simple, efficient, and works well for linearly separable data.

Linear SVC: Linear Support Vector Classifier (Linear SVC) is a powerful classification algorithm that finds an optimal hyperplane to separate different classes. It is particularly effective for high-dimensional data such as text represented using TF-IDF vectors.

5. Model Performance Visualization

To compare the performance of the models, a graphical representation of their accuracies is shown in Figure 3.

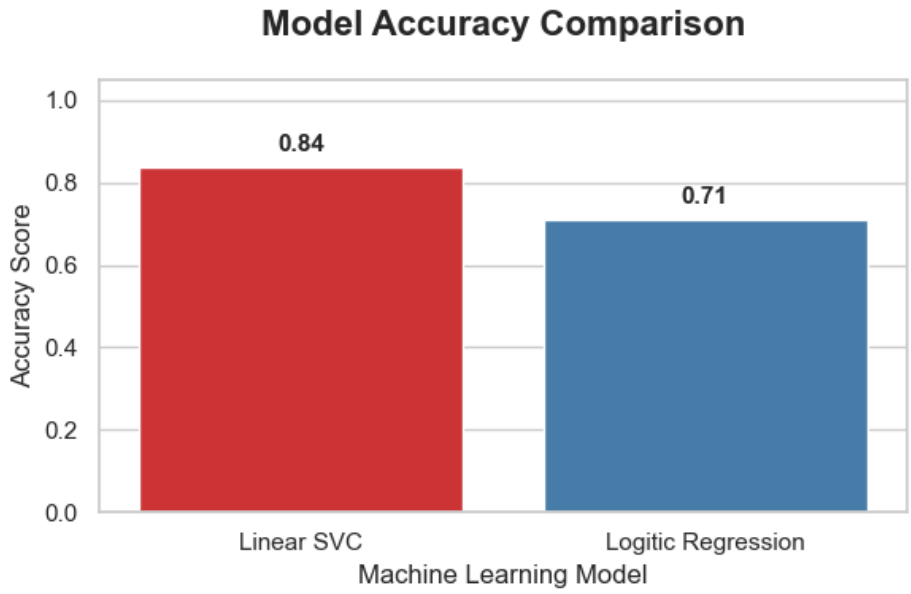

Figure 3: Comparison of Logistic Regression and Linear SVC based on accuracy.

6. Output to Action Module

Once the model predicts the intent of the user command, the predicted label is passed to the **Action Router**, which maps the intent to the corresponding system module for execution.

This approach ensures efficient and accurate understanding of user commands, enabling seamless interaction between the user and the CORTEX system.

—

3.4 Action Module: Command Execution and Task Automation

Once the intent is predicted, the system transitions from decision-making to execution. The **Action Module** is responsible for mapping the predicted intent to a specific operation and performing the corresponding task on the system.

This is achieved through a centralized component called the **Action Router**, which connects the machine learning output to real-world system actions.

1. Action Routing Mechanism

The Action Router maintains a mapping between predicted intents and their corresponding action modules using a dictionary structure (`intent_map`). Each intent is associated with a dedicated Python class responsible for executing that task.

A key design principle of this system is that **all action modules implement a common function:** `get_response(command)`.

This uniform function design provides:

- **Consistency:** All actions are executed using the same method call.
- **Scalability:** New actions can be added without modifying the routing logic.
- **Flexibility:** Enables easy implementation of fallback mechanisms.

Once an intent is predicted, the router directly invokes:

```
action.get_response(command)
```

Each action processes the command and returns a response along with an optional action or process made for the specific command.

2. Action Categories and Implementation

The system supports multiple categories of actions, each implemented using specialized Python libraries:

- **System Control:** Includes operations such as volume control, brightness adjustment, screen locking, restart, and shutdown.
  - Libraries: `pycaw`, `screen_brightness_control`, `os`, `subprocess`, `pywin32`
- **System Monitoring:** Provides real-time system information such as battery status, disk usage, uptime, Wi-Fi status, and IP address.
  - Libraries: `psutil`, `socket`, `WMI`
- **Application Control:** Allows opening desktop applications through system-level execution.
  - Libraries: `subprocess`, `os`
- **File and Search Operations:** Supports searching and accessing documents, PDFs, images, and folders using both exact and approximate matching techniques.
  - Libraries: `os`, `glob`, `rapidfuzz`
  - File traversal is performed using `os.walk()`.

  - `rapidfuzz` is used for fuzzy string matching, allowing the system to handle partial or misspelled file names effectively.

- **Internet and Web Actions:** Handles opening websites and basic online interactions.
  - Libraries: `webbrowser`, `requests`
- **Utilities:** Includes clipboard reading, date-time retrieval, and screenshot capture.
  - Libraries: `pyperclip`, `datetime`, `pyautogui`
- **Media Control:** Handles multimedia operations such as play, pause, and navigation.
  - Libraries: `pyautogui`
- **Basic Interaction:** Includes greeting responses and controlled system exit.

Each module is independently designed and encapsulates its functionality, ensuring modularity and maintainability.

3. Fallback Mechanism

In cases where the predicted intent has low confidence or is not mapped correctly, the system activates a fallback mechanism.

The Action Router iterates through all available action modules and invokes their `get_response()` methods to identify a possible match.

This improves:

- Robustness against ambiguous commands
- Reliability of the system

4. Integration with Main Window

The `main_window.py` serves as the **central orchestration layer** of the system, coordinating interaction between voice input, authentication, intent processing, action execution, and the graphical user interface.

Unlike a simple sequential program, the system is designed using **multi-threading and event-driven architecture (PyQt5)**, ensuring that all components operate concurrently without blocking the user interface.

System Initialization and Activation:

At startup, the main window initializes the Action Router, UI components, and internal state variables. The system then transitions into an active state using timed triggers (`QTimer`), ensuring a smooth startup sequence.

- Initializes `ActionRouter` for intent-action mapping.
- Loads the animated UI (`JarvisUI`).
- Activates system readiness after a short delay.

Security and Authentication Flow:

Before allowing any interaction, the system enforces a **voice-based authentication mechanism**. This is implemented using separate threads to avoid blocking execution:

- **WaitForEnterThread:** Waits for user input to start recording.
- **VerifyThread:** Performs voice authentication using `verify_voice()`.

If authentication fails multiple times, a fallback password dialog is triggered. Upon failure, the system enforces security by locking the workstation using system-level APIs.

Voice Input and Asynchronous Processing:

User commands are captured using a dedicated thread:

- **ListenThread:** Captures voice input using `SpeechRecognition` (Google Listerner for normal commands and Faster-Whisper for Voice Authentication) and converts it into text.

This ensures that the GUI remains responsive while the system listens for input. The recognized text is emitted using PyQt signals and passed to the main processing pipeline.

Intent Processing and Action Execution:

Once the command is received, it is forwarded to the Action Router:

```
response, action = action_router.action_handler(command)
```

Based on the output:

- The response is converted to speech using a non-blocking TTS system (`subprocess`-based execution).
- The UI state is updated dynamically (**listening, speaking, idle**).

- If an executable function is returned, it is triggered directly.

Dynamic UI Management:

The system supports multiple UI modes to ensure clear separation between offline system actions and internet-based functionalities:

- **Normal Mode:** Standard assistant interface (`JarvisUI`) used for executing system-level commands such as application control, file operations, and system monitoring.
- **Internet Mode:** Specialized interface (`Internet_UI`) designed for handling dynamic online queries and real-time information retrieval.

5. Internet Mode

The transition between these modes is managed dynamically by the main window based on specific action signals (e.g., `internet_on`, `internet_off`) without requiring application restart. This ensures a seamless user experience while maintaining architectural separation.

The Internet Mode is powered using the **Groq API**, which enables the system to handle a wide range of online tasks such as general queries, information retrieval, and conversational responses. Unlike predefined intent-based actions, this mode allows flexible and context-aware interactions.

To prevent conflicts between machine learning-based intent execution and open-ended internet queries, the system strictly separates both functionalities into two distinct modes. This design avoids ambiguity and ensures that system commands are not misinterpreted as internet queries and vice versa.

Mode switching is controlled using explicit trigger phrases:

- **"Internet mode"** – Activates Internet Mode
- **"Normal mode"** – Returns to standard system operation

Additionally, a periodic timer is implemented using `QTimer` to continuously bring the active UI to the foreground. This ensures that the assistant interface remains visible and accessible during interaction, even when other applications are in use.

6. Event Handling and User Interaction:

User interaction is designed to be seamless and responsive:

- **Keyboard triggers (`Right Ctrl`) initiate listening.**
- UI states visually indicate system activity.

- Signals and slots (PyQt mechanism) handle communication between threads and UI.

Concurrent Execution Design:

The system is designed to handle multiple operations simultaneously without affecting responsiveness. This is achieved using a combination of multi-threading and event-driven programming provided by `PyQt5`.

- **QThread:** Dedicated threads are used for time-consuming operations such as voice listening (`ListenThread`) and authentication (`VerifyThread`). By offloading these tasks to separate threads, the main UI thread remains unblocked and responsive at all times.
- **Signals and Slots:** PyQt's signal-slot mechanism is used for safe and efficient communication between threads. For example, once voice input is processed in a background thread, the result is emitted as a signal and handled by the main window for further processing. This avoids direct thread interference and ensures thread safety.
- **QTimer:** Timers are used to schedule tasks such as system initialization, delayed execution, UI state transitions, and periodic updates (e.g., keeping the UI always on top). This allows precise control over execution flow without blocking the main thread.

<u>This ensures:</u>

- **Non-blocking Voice Input:** The system can continuously listen for user commands while maintaining an active and responsive interface.
- **Smooth UI Performance:** Animations and visual feedback in the GUI are not interrupted by background processing tasks.
- **Real-time Processing:** Commands are processed and executed immediately after recognition, ensuring minimal latency in response generation.
- **Parallel Task Handling:** Multiple operations such as listening, authentication, and action execution can occur concurrently without conflicts.

Overall, the `main_window.py` integrates all subsystems into a unified and efficient execution pipeline. By combining multi-threading, event-driven communication, and modular design, the system delivers a secure, responsive, and intelligent user experience while maintaining stability and scalability.

The complete application lifecycle is managed through the `main.py` entry point, which initializes the `QApplication`, launches the main window, and starts the event loop. This ensures proper execution flow, resource management, and seamless interaction between all components of the CORTEX assistant. Figure 4 illustrates the architecture, showing the integration of the Action Router, Voice Processing, and Voice Authentication modules with `main_window.py`, and their connection to `main.py`.

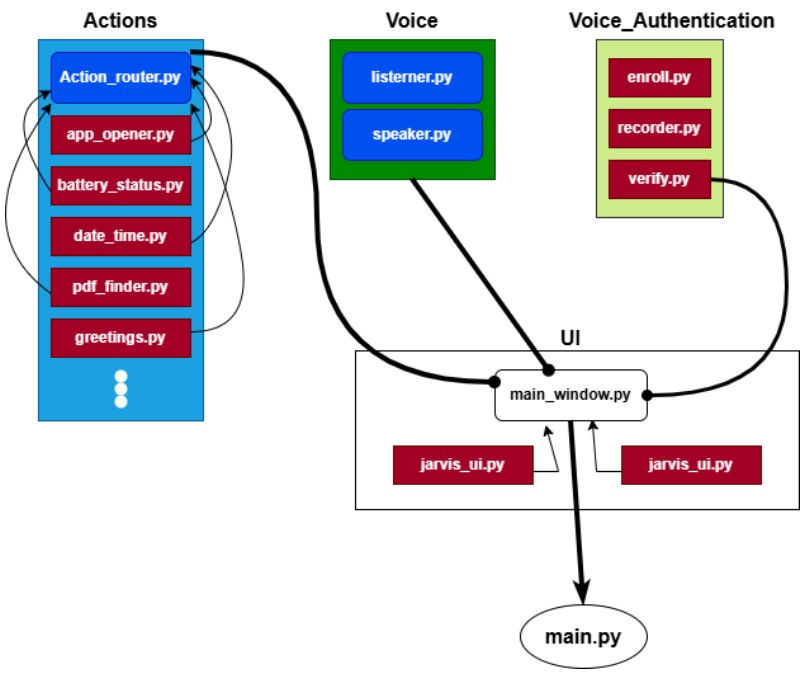

Figure 4: Action Router Architecture and Integration of all modules with Main Window.

7. Security and Execution Control

All system-level actions are executed with controlled access. Critical operations such as shutdown and restart are triggered only through explicit user commands. This ensures safe and secure behavior while interacting with core system. —

3.5 User Interface: Desktop UI Design and Interaction

The CORTEX desktop interface is developed using **PyQt5**, providing a visually interactive and responsive environment for user interaction. The UI is designed not only for aesthetics but also to clearly reflect the internal state of the system in real time.

1. UI Architecture and Components

The system consists of two primary user interface modules:

- **Jarvis UI:** The default interface used in Normal Mode for system-level interactions such as executing commands, displaying responses, and showing assistant states.
- **Internet UI:** A specialized interface activated during Internet Mode, optimized for handling online queries and dynamic responses powered by the Groq API.

The main window dynamically switches between these interfaces based on system mode, ensuring a seamless and uninterrupted user experience.

2. Dynamic State Representation

The UI is designed to visually represent the assistant's current state using animations and transitions. The system operates in three primary states:

- **Listening State:** Activated when the system is capturing user input. The UI displays animated indicators (e.g., pulsing or waveform effects) to signal that the assistant is actively listening.
- **Processing/Speaking State:** Triggered when the system is generating or delivering a response. The UI reflects this through distinct animations, indicating that the assistant is responding to the user.

- **Idle State:** The default resting state where the system is ready for input. The UI remains minimal and stable, ensuring clarity and low visual distraction.

These states are controlled programmatically through functions such as `set_state()`, allowing seamless transitions based on system activity.

3. Real-Time Interaction and Feedback

The interface provides immediate visual and auditory feedback to enhance usability:

- Displays system responses in sync with voice output
- Reflects command execution status through UI changes
- Ensures smooth transitions between listening, processing, and idle states

This real-time feedback loop improves user understanding of system behavior and creates a more natural interaction experience.

4. User Interaction and Control

User interaction is designed to be simple and efficient:

- **Keyboard triggers (e.g., Right Ctrl) initiate voice listenin**g
- UI remains always on top to ensure accessibility
- Visual cues guide the user during interaction (e.g., when to speak)

The integration of keyboard input and visual feedback allows quick and intuitive control of the assistant.

5. Design Considerations and Scalability

The UI is built with a modular and extensible design, allowing future enhancements such as:

- Task scheduling and reminders
- Notification systems
- Advanced conversational interfaces
- Integration with additional AI modules

The separation of UI components (Jarvis UI and Internet UI) ensures that new features can be added without affecting existing functionality. Figure 5 shows the transaction of Jarvis UI and Internet UI.

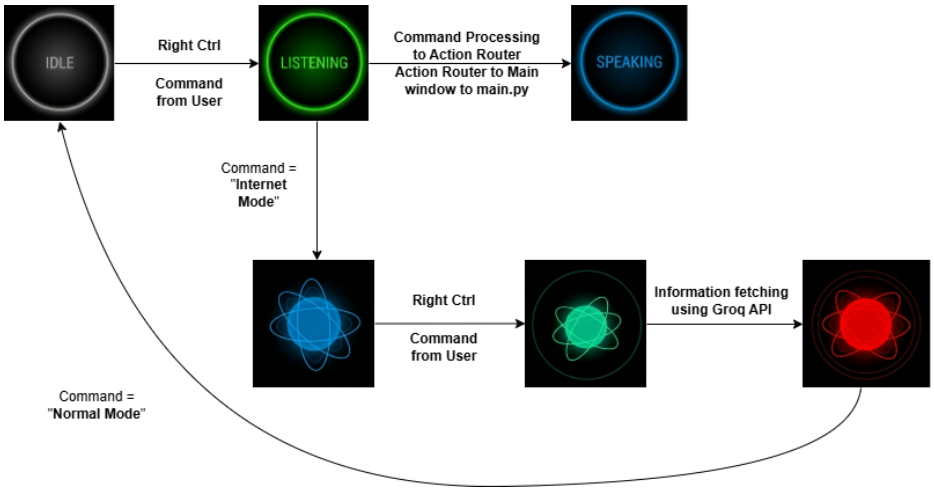

Figure 5: CORTEX Desktop UI showing user input and visual feedback.

6. Overall User Experience

The combination of animated feedback, real-time responsiveness, and clear system states results in an intuitive and engaging user experience. The UI not only acts as a visual layer but also as a communication bridge between the user and the intelligent backend of the CORTEX system.

4. RESULTS AND ANALYSIS :

System Performance Evaluation

The performance of the CORTEX voice assistant system was evaluated based on its ability to accurately recognize user commands, predict intents, and execute corresponding actions in real time. The evaluation focuses on three key components: **speech recognition**, **intent prediction**, and **action execution**.

1. Speech Recognition Performance

The system uses **SpeechRecognition** along with **faster-whisper** for converting voice input into text. The performance was evaluated based on clarity, background noise handling, and response time.

- High accuracy for clear speech in controlled environments
- Moderate robustness in presence of background noise
- Average transcription latency of **1–2 seconds**

The integration of Whisper-based models significantly improves transcription quality compared to traditional speech recognition approaches.

2. Intent Prediction Accuracy

The intent prediction module, based on **TF-IDF vectorization** and **Linear SVC**, demonstrated strong performance in classifying user commands.

- Achieved an accuracy of approximately **84%** on the test dataset
- Efficient handling of short and structured commands
- Reliable mapping of user input to predefined intent categories

The model performs well for known command patterns but may require additional training data for highly varied natural language inputs.

3. Action Execution Performance

The Action Module successfully executed a wide range of system-level and application-level commands. Performance was evaluated based on execution speed and reliability.

- System commands (volume, brightness, lock, shutdown) executed instantly
- File search operations showed slight delay depending on directory size
- Application launching and web operations performed reliably

The modular architecture ensured that each action executed independently without affecting overall system stability.

4. Real-Time System Responsiveness

The integration of **multi-threading (QThread)** and **event-driven programming (PyQt5)** ensured smooth real-time interaction.

- No UI freezing during voice input or processing
- Simultaneous handling of listening, processing, and UI updates
- Fast response generation with minimal delay

This confirms the effectiveness of the concurrent execution design implemented in the system.

5. Internet Mode Performance

The Internet Mode, powered by the **Groq API**, enables the system to handle open-ended queries beyond predefined intents.

- Successfully processes general knowledge and conversational queries
- Provides context-aware and dynamic responses
- Seamless switching between Normal Mode and Internet Mode using trigger phrases

The separation of Internet Mode from Normal Mode ensures that machine learning-based commands do not conflict with AI-based responses.

6. User Interface Evaluation

The PyQt5-based UI was evaluated based on usability and responsiveness:

- Clear visual representation of system states (Listening, Speaking, Idle)
- Smooth animations and transitions between states
- Always-on-top interface ensures accessibility

The UI enhances user interaction by providing immediate visual feedback and improving overall usability.

7. Security and Authentication Evaluation

The voice authentication mechanism ensures secure access to the system:

- Successful verification using voice-based matching
- Multi-attempt handling with fallback to password authentication
- System lock triggered after repeated failures

This layered security approach improves system reliability and prevents unauthorized access.

8. Overall System Analysis

The CORTEX system demonstrates:

- Efficient integration of machine learning and system automation
- Robust handling of real-time voice interactions
- Scalable architecture for future enhancements

While the system performs effectively for predefined commands and structured queries, future improvements can focus on enhancing natural language understanding and expanding the dataset for better generalization.

5. FUTURE SCOPE AND CONCLUSION :

The CORTEX Desktop Voice Assistant successfully demonstrates how voice recognition, machine learning, and system automation can be integrated to simplify human-computer interaction. By enabling users to perform tasks through natural voice commands, the system reduces manual effort and enhances productivity.

The project highlights the practical implementation of an intelligent assistant capable of handling real-time commands, executing system-level operations, and providing dynamic responses through both predefined intents and AI-powered internet queries. The modular architecture, combined with concurrent execution and an interactive user interface, ensures a responsive and scalable system.

Currently, the system is designed to run within the **VS Code terminal environment**, which serves as a development and testing platform. However, this is a foundational step toward building

a more accessible solution. In future, the system will be developed into a **standalone desktop application**, allowing users to install and use it easily without requiring a development setup.

Future Scope:

- **Standalone Application Development:** Packaging the system into an installable desktop application for wider accessibility.
- **User Customization Interface:** Integration of a user-friendly configuration system (e.g., Google Form or in-app settings) where users can:
  - Enter and manage personal details
  - Switch between **male and female voice output**
  - Enroll their voice for authentication
  - Customize assistant behavior according to preferences
- **Advanced AI Integration:** Enhancing conversational capabilities by integrating more advanced AI chatbot systems for deeper and more natural interactions.
- **Face Recognition Authentication:** Adding an additional layer of security using computer vision for user verification.
- **Smart Home Integration (IoT):** Enabling control of IoT devices such as lights, fans, and appliances through voice commands.
- **Mobile Application Integration:** Extending the assistant to mobile platforms for remote accessibility and control.
- **Multi-language Support:** Supporting multiple languages to improve accessibility and usability for a broader audience.

Conclusion:

Overall, the CORTEX system establishes a robust foundation for the development of next-generation intelligent voice assistants by effectively integrating speech recognition, machine learning, system automation, and real-time user interaction within a unified architecture. The project demonstrates how voice-based interfaces can significantly enhance human-computer interaction by making it more natural, efficient, and accessible.

Through its modular design, concurrent execution, and dual-mode functionality (Normal and Internet Mode), the system not only handles predefined system tasks with high reliability but also supports dynamic, AI-driven interactions. The incorporation of security mechanisms such as voice authentication further strengthens its practicality for real-world applications.

As technology continues to advance, systems like CORTEX have the potential to evolve into fully personalized digital assistants capable of adapting to individual user preferences, environments, and behaviors. This work highlights the growing importance of intelligent automation and serves as a stepping stone toward more secure, scalable, and widely accessible AI-powered computing solutions.

6. REFERENCES :

References

[1] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," in *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257-286, Feb. 1989, doi: 10.1109/5.18626.

[2] Gowthamy, J Ayothi, Senthilselvi Kumar, Aniket Aakash, S Sreedhar, Gandikota. (2023). Enhanced AI Voice Assistance using Machine Learning and NLP. 1-5. 10.1109/STCR59085.2023.10396893.

[3] U. Adha, I. Tinna, S. Singh, A. Kumar, and Chanchal, "Jarvis: Virtual Voice Command Desktop Assistant," *International Journal of Innovative Science and Research Technology (IJISRT)*, vol. 8, no. 11, pp. 1675–1678, Nov. 2023.

[4] H. Asodariya, K. Vachhani, E. Ghori, B. Babariya, and T. Patel, "Desktop Voice Assistant," *International Journal of Innovative Science and Research Technology (IJISRT)*, vol. 8, no. 2, pp. 1418–1420, Feb. 2023.

[5] A. Graves, A. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *2013 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Vancouver, BC, Canada, 2013, pp. 6645–6649, doi: 10.1109/ICASSP.2013.6638947.

[6] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed., Pearson, 2023.

[7] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[8] A. Radford et al., "Whisper: Robust speech recognition via large-scale weak supervision," *OpenAI*, 2022.

[9] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[10] F. Chollet, *Deep Learning with Python*, 2nd ed., Manning Publications, 2021.

SRINATH NEOGI

8th_Sem_Project_Report_Project_CORTEX

Student Assignment

Document Details

Submission ID

trn:oid:::3618:133893806

Submission Date

Apr 1, 2026, 8:02 PM GMT+5:30

Download Date

Apr 1, 2026, 8:06 PM GMT+5:30

File Name

8th_Sem_Project_Report_Project_CORTEX.pdf

File Size

418.5 KB

34 Pages

6,291 Words

39,470 Characters

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 10 words)

Match Groups

16 Not Cited or Quoted 4%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
2% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **16** Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
2% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.coursehero.com	<1%
2	Internet	de.overleaf.com	<1%
3	Internet	www.ijraset.com	<1%
4	Internet	1library.net	<1%
5	Internet	instadatahelp.com	<1%
6	Publication	J Gowthamy, A Senthilselvi, Aniket Kumar, S Aakash, Gandikota Sreedhar. "Enhan...	<1%
7	Internet	download.bibis.ir	<1%
8	Internet	ebin.pub	<1%
9	Internet	shga.kr	<1%
10	Internet	www.ijert.org	<1%